

Math for AI/ML — Complete Guide

Tumare liye specially crafted | Python Developer Edition

Tera Background: Python | JS/MERN | n8n/Automations

Goal: AI/ML ke liye math ka solid foundation banana

Pehle Samjho — Kyun Zaroori Hai Yeh Math?

Math Topic	AI/ML mein Use	Weight
Probability & Statistics	Model uncertainty, predictions, data analysis	~50%
Linear Algebra	Data representation, neural networks, transformations	~35%
Calculus	Model training (gradient descent), optimization	~15%

Simple analogy: Jaise MERN stack ke liye JavaScript zaroori hai, waise AI/ML ke liye yeh teen pillars zaroori hain.

PART 1: PROBABILITY & STATISTICS

Complete Foundation

1.1 Basic Probability — Shuruat Yahan Se

Core Concepts

Sample Space (Ω): Saare possible outcomes ka set

Event (A): Sample space ka ek subset

Probability: $P(A) = (\text{Favorable outcomes}) / (\text{Total outcomes})$

Rule 1: $0 \leq P(A) \leq 1$

Rule 2: $P(\Omega) = 1$ (koi na koi outcome zaroor aayega)

Rule 3: $P(\emptyset) = 0$ (impossible event)

Key Formulas

$P(A \cup B) = P(A) + P(B) - P(A \cap B)$ ← Addition Rule
 $P(A^c) = 1 - P(A)$ ← Complement Rule
 $P(A \cap B) = P(A) \times P(B)$ ← Only if A, B are INDEPENDENT

Python Code

```
import numpy as np

# Simulate 10,000 fair coin flips
flips = np.random.choice(['H', 'T'], size=10000)
p_heads = np.sum(flips == 'H') / len(flips)
print(f"P(Heads) ≈ {p_heads:.4f}") # Should be ~0.5
```

```
# Dice example: P(even number)
dice_rolls = np.random.randint(1, 7, size=100000)
p_even = np.sum(dice_rolls % 2 == 0) / len(dice_rolls)
print(f"P(Even) ≈ {p_even:.4f}") # Should be ~0.5

# P(A ∪ B): P(even OR > 4)
p_a = np.sum(dice_rolls % 2 == 0) / len(dice_rolls) # P(even)
p_b = np.sum(dice_rolls > 4) / len(dice_rolls) # P(>4)
p_a_and_b = np.sum((dice_rolls % 2 == 0) & (dice_rolls > 4)) / len(dice_rolls)
p_a_or_b = p_a + p_b - p_a_and_b
print(f"P(Even OR >4) ≈ {p_a_or_b:.4f}") # True answer: 4/6 ≈ 0.667
```

1.2 Conditional Probability

Definition: B hone KE BAAD A hone ki probability

$$P(A|B) = P(A \cap B) / P(B)$$

Real ML Example: Spam detection mein —

$P(\text{spam} \mid \text{email contains "lottery"}) = ?$

```
# Medical Test Example
# Disease prevalence: 1%
# Test accuracy: 99% sensitivity, 95% specificity

p_disease = 0.01 # P(D)
p_no_disease = 0.99 # P(D')
p_pos_given_disease = 0.99 # P(+|D) - sensitivity
p_pos_given_no_disease = 0.05 # P(+|D') - false positive rate

# P(Positive test) using Total Probability
p_positive = (p_pos_given_disease * p_disease) + \
             (p_pos_given_no_disease * p_no_disease)

print(f"P(Positive) = {p_positive:.4f}")
```

1.3 Bayes' Theorem — AI ka Dil

Yeh ML ka SABSE important formula hai.

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

Where:

```
P(A)      = Prior (pehle se kya pata tha)
P(B|A)    = Likelihood (evidence A ke hone par)
P(B)      = Evidence (normalizing constant)
P(A|B)    = Posterior (evidence dekhne ke baad updated belief)
```

Naive Bayes Classifier isi pe based hai!

```
# Bayes Theorem: Disease Detection
p_disease = 0.01
p_pos_given_disease = 0.99
p_pos_given_no_disease = 0.05

# P(Disease | Positive Test) = ?
numerator = p_pos_given_disease * p_disease
denominator = (p_pos_given_disease * p_disease) + \
              (p_pos_given_no_disease * (1 - p_disease))

p_disease_given_positive = numerator / denominator
```

```

print(f"P(Disease | Positive) = {p_disease_given_positive:.4f}")
# Result: Only ~16.7% even with a positive test!
# Yahi Bayes ka magic hai — intuition ko challenge karta hai

# Simple Bayes from scratch using sklearn
from sklearn.naive_bayes import GaussianNB
import numpy as np

X_train = np.array([[1, 0], [1, 1], [0, 1], [0, 0]]) # features
y_train = np.array([1, 1, 0, 0]) # labels
model = GaussianNB()
model.fit(X_train, y_train)
print(model.predict([[1, 0]])) # Predict new sample

```

1.4 Combinatorics (Counting)

Permutation: $nPr = n! / (n-r)!$ ← Order MATTERS
Combination: $nCr = n! / [r!(n-r)!]$ ← Order does NOT matter

ML mein use: Hyperparameter combinations, feature selection

```

from math import factorial, comb, perm

# Choose 3 features from 10 features
n, r = 10, 3
combinations = comb(n, r)
print(f"C(10,3) = {combinations}") # 120 ways

# Permutations
permutations = perm(n, r)
print(f"P(10,3) = {permutations}") # 720 ways

# Real use: How many hyperparameter combos?
# learning_rates = [0.001, 0.01, 0.1] → 3 options
# batch_sizes = [32, 64, 128] → 3 options
# layers = [2, 3, 4] → 3 options
total_combos = 3 * 3 * 3
print(f"Grid search combinations: {total_combos}") # 27

```

1.5 Descriptive Statistics — Data ko Samajhna

Measures of Central Tendency

Measure	Formula	Use in ML
Mean	$\Sigma x / n$	Feature scaling, baseline
Median	Middle value	Outlier-robust mean
Mode	Most frequent	Categorical data

Measures of Spread

Measure	Formula	Use in ML
Variance	$\Sigma (x-\mu)^2 / n$	Model complexity
Std Dev	$\sqrt{\text{Variance}}$	Feature normalization
Range	max - min	Outlier detection

```

import numpy as np
import pandas as pd

```

```

data = np.array([2, 4, 4, 4, 5, 5, 7, 9, 10, 100]) # Note: 100 is outlier

print("=== CENTRAL TENDENCY ===")
print(f"Mean: {np.mean(data):.2f}") # Affected by outlier
print(f"Median: {np.median(data):.2f}") # NOT affected by outlier
# Mode manually (numpy doesn't have direct mode)
from scipy import stats
print(f"Mode: {stats.mode(data).mode}")

print("\n=== SPREAD ===")
print(f"Variance: {np.var(data):.2f}")
print(f"Std Dev: {np.std(data):.2f}")
print(f"Range: {np.max(data) - np.min(data)}")

print("\n=== PERCENTILES ===")
print(f"25th percentile (Q1): {np.percentile(data, 25)}")
print(f"75th percentile (Q3): {np.percentile(data, 75)}")
print(f"IQR: {np.percentile(data, 75) - np.percentile(data, 25)}")

```

1.6 Probability Distributions — ML ka Bread & Butter

Discrete Distributions

Binomial Distribution: n trials mein exactly k successes

```

P(X=k) = C(n,k) × p^k × (1-p)^(n-k)

Mean = np
Variance = np(1-p)

from scipy import stats
import matplotlib.pyplot as plt
import numpy as np

# Binomial: 10 coin flips, p(head)=0.5, P(exactly 6 heads)?
n, p = 10, 0.5
k = 6
prob = stats.binom.pmf(k, n, p)
print(f"P(X=6) = {prob:.4f}") # ~0.205

# Plot PMF
x = np.arange(0, 11)
pmf = stats.binom.pmf(x, n, p)
plt.bar(x, pmf)
plt.xlabel('k (number of heads)')
plt.ylabel('P(X=k)')
plt.title('Binomial Distribution (n=10, p=0.5)')
plt.savefig('binomial.png')

```

Continuous Distributions

Normal (Gaussian) Distribution — Sabse Important!

$$f(x) = (1/\sigma\sqrt{2\pi}) \times e^{-(x-\mu)^2/2\sigma^2}$$

68% data within 1σ of mean
 95% data within 2σ of mean
 99.7% data within 3σ of mean

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

```

```

# Standard Normal ( $\mu=0$ ,  $\sigma=1$ )
x = np.linspace(-4, 4, 1000)
pdf = stats.norm.pdf(x, loc=0, scale=1)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(x, pdf, 'b-', linewidth=2)
plt.fill_between(x, pdf, where=(abs(x) <= 1), alpha=0.3, label='68%')
plt.fill_between(x, pdf, where=(abs(x) <= 2), alpha=0.2, label='95%')
plt.title('Normal Distribution PDF')
plt.legend()

# Z-Score: koi bhi value ko standard scale pe laana
#  $z = (x - \mu) / \sigma$ 
data_point = 170 # height in cm
mu = 165 # population mean
sigma = 10 # population std

z_score = (data_point - mu) / sigma
print(f"Z-score: {z_score}") # 0.5 (half std above mean)
percentile = stats.norm.cdf(z_score) * 100
print(f"Percentile: {percentile:.1f}%") # 69.1%

```

Poisson Distribution

Count of rare events in fixed interval (e.g., errors per hour)

```

P(X=k) = ( $\lambda^k \times e^{-\lambda}$ ) / k!
Mean =  $\lambda$ , Variance =  $\lambda$ 

# Average 3 customer calls per hour ( $\lambda=3$ )
# P(exactly 5 calls in next hour)?
lam = 3
k = 5
prob = stats.poisson.pmf(k, lam)
print(f"P(5 calls) = {prob:.4f}") # ~0.101

```

1.7 Covariance & Correlation — Relationships Between Variables

Covariance: $\text{Cov}(X, Y) = E[(X - \mu_x)(Y - \mu_y)]$
 Correlation: $\rho = \text{Cov}(X, Y) / (\sigma_x \times \sigma_y)$

$\rho = +1$: Perfect positive relationship
 $\rho = 0$: No relationship
 $\rho = -1$: Perfect negative relationship

ML mein use: Feature selection, multicollinearity detection

```

import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Synthetic dataset
np.random.seed(42)
n = 100
X1 = np.random.randn(n) # Feature 1
X2 = 2 * X1 + np.random.randn(n) # Correlated with X1
X3 = np.random.randn(n) # Independent

data = np.column_stack([X1, X2, X3])
corr_matrix = np.corrcoef(data.T)

```

```

print("Correlation Matrix:")
print(np.round(corr_matrix, 2))

# Heatmap
plt.figure(figsize=(6, 4))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm',
            xticklabels=['X1', 'X2', 'X3'],
            yticklabels=['X1', 'X2', 'X3'])
plt.title('Correlation Matrix')
plt.savefig('correlation.png')

```

1.8 Hypothesis Testing — Statistical Decision Making

ML mein experiments test karne ke liye (A/B testing, model comparison)

Core Concepts

- **Null Hypothesis (H₀):** "Koi fark nahi" — default assumption
- **Alternate Hypothesis (H₁):** "Fark hai" — jo tum prove karna chahte ho
- **p-value:** Agar H₀ sach hai, toh observed data itna extreme hone ki probability
- **α (significance level):** Usually 0.05 — rejection threshold

If p-value < α → Reject H₀ (statistically significant)
 If p-value ≥ α → Fail to reject H₀

```

from scipy import stats
import numpy as np

```

```

# T-test: Kya do models ki accuracy alag hai?
model_a_scores = np.array([0.82, 0.85, 0.83, 0.87, 0.84])
model_b_scores = np.array([0.88, 0.90, 0.89, 0.91, 0.87])

t_stat, p_value = stats.ttest_ind(model_a_scores, model_b_scores)
print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

```

```

alpha = 0.05
if p_value < alpha:
    print("☑ Significant difference! Model B is better.")
else:
    print("☐ No significant difference found.")

```

```

# Chi-Square Test: Categorical data ke liye
# Is there a relationship between device type and purchase?
observed = np.array([[30, 10], # Mobile: bought, not bought
                    [15, 25]]) # Desktop: bought, not bought
chi2, p, dof, expected = stats.chi2_contingency(observed)
print(f"\nChi2 = {chi2:.3f}, p = {p:.4f}")

```

☐ PART 2: LINEAR ALGEBRA

The Language of Data

2.1 Vectors — Data ka Basic Unit

Yeh samjho: Ek data point = ek vector

A user with [age=25, income=50000, purchases=3] = vector [25, 50000, 3]

```
Column vector: [x1]   Row vector: [x1, x2, x3]
               [x2]
               [x3]
```

Vector Operations

```
import numpy as np

# Vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Addition
print("a + b =", a + b)          # [5, 7, 9]

# Scalar multiplication
print("3 × a =", 3 * a)          # [3, 6, 9]

# DOT PRODUCT — Sabse important!
# a·b = a1b1 + a2b2 + a3b3
dot = np.dot(a, b)
print("a·b =", dot)              # 4+10+18 = 32

# Magnitude (L2 Norm)
mag_a = np.linalg.norm(a)
print("||a|| =", mag_a)          # √(1+4+9) = √14

# Unit vector (normalize)
unit_a = a / np.linalg.norm(a)
print("unit vector:", unit_a)

# Cosine Similarity — Used in NLP/Recommendation systems!
cos_sim = np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print(f"Cosine Similarity: {cos_sim:.4f}") # 0.9746 (very similar)
```

Why Dot Product Matters in ML?

```
In Neural Networks: y = w·x + b
w = weight vector
x = input vector
b = bias
```

This is literally how every neuron works!

2.2 Matrices — Data Tables ka Math Form

Yeh samjho: Dataset = Matrix

100 rows, 5 features → 100×5 Matrix

```
import numpy as np

# Create matrices
A = np.array([[1, 2, 3],
              [4, 5, 6]]) # 2×3 matrix

B = np.array([[7, 8],
              [9, 10],
              [11, 12]]) # 3×2 matrix
```

```

# Matrix Multiplication: A(2×3) × B(3×2) = C(2×2)
# Rule: Columns of A must equal Rows of B
C = np.dot(A, B)          # or A @ B
print("A × B =\n", C)

# Transpose: Rows ↔ Columns
print("A transpose:\n", A.T)    # Shape: 3×2

# Identity Matrix
I = np.eye(3)
print("Identity:\n", I)

# Inverse Matrix (square matrices only)
M = np.array([[2, 1],
              [5, 3]])
M_inv = np.linalg.inv(M)
print("M inverse:\n", M_inv)
print("M × M_inv ≈ I:\n", np.round(M @ M_inv))

# Determinant (scalar value, tells if inverse exists)
det = np.linalg.det(M)
print(f"Det(M) = {det:.1f}")    # Non-zero → invertible

```

2.3 Solving Linear Equations — ML ka Core

```

In ML (Linear Regression):
 $X\theta = y$ 
 $\theta = (X^T X)^{-1} X^T y$  ← Normal Equation

import numpy as np

# System: 2x + y = 5, x + 3y = 10
A = np.array([[2, 1],
              [1, 3]])
b = np.array([5, 10])

# Method 1: np.linalg.solve
x = np.linalg.solve(A, b)
print(f"x = {x[0]:.2f}, y = {x[1]:.2f}")    # x=1, y=3

# Linear Regression using Normal Equation
np.random.seed(42)
n_samples = 50
X_data = 2 * np.random.randn(n_samples, 1)
y_data = 4 + 3 * X_data + np.random.randn(n_samples, 1)

# Add bias term (column of 1s)
X_b = np.c_[np.ones((n_samples, 1)), X_data]

# Normal equation:  $\theta = (X^T X)^{-1} X^T y$ 
theta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y_data
print(f"\nIntercept ( $\theta_0$ ): {theta[0][0]:.2f}")    # Should be ~4
print(f"Slope ( $\theta_1$ ): {theta[1][0]:.2f}")    # Should be ~3

```

2.4 Eigenvalues & Eigenvectors — PCA ka Dil

Concept: Ek special vector jo matrix transformation ke baad sirf scale hota hai (direction nahi badlta)

$Av = \lambda v$
A = matrix, v = eigenvector, λ = eigenvalue

ML mein use: PCA (Dimensionality Reduction), Graph algorithms, PageRank


```

import numpy as np

A = np.array([[3, 1],
              [1, 3]])

# Find eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
print("Eigenvalues:", eigenvalues)          # [4, 2]
print("Eigenvectors:\n", eigenvectors)

# Verify: Av = λv
v = eigenvectors[:, 0]      # First eigenvector
lam = eigenvalues[0]        # First eigenvalue
print("\nAv =", A @ v)
print("λv =", lam * v)     # Should be same!

# PCA using eigenvalues (simplified)
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

iris = load_iris()
X = iris.data

# Standardize
X_scaled = StandardScaler().fit_transform(X)

# PCA to 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print(f"\nOriginal shape: {X.shape}")      # (150, 4)
print(f"After PCA shape: {X_pca.shape}")    # (150, 2)
print(f"Variance explained: {pca.explained_variance_ratio_}")

```

2.5 SVD — Singular Value Decomposition

Advanced concept lekin bahut useful:

$$A = U \times \Sigma \times V^T$$

U = left singular vectors
 Σ = diagonal matrix of singular values
 V^T = right singular vectors transposed

Use in ML: Recommendation systems (Netflix!), image compression, NLP

```

import numpy as np
import matplotlib.pyplot as plt

# Image compression using SVD
# Simulate a matrix (like a grayscale image)
np.random.seed(42)
image = np.random.randint(0, 256, (50, 50)).astype(float)

# SVD decomposition
U, sigma, Vt = np.linalg.svd(image)

# Reconstruct with only top-k singular values
def reconstruct(U, sigma, Vt, k):
    return U[:, :k] @ np.diag(sigma[:k]) @ Vt[:k, :]

for k in [5, 10, 20, 50]:
    reconstructed = reconstruct(U, sigma, Vt, k)

```

```
error = np.mean((image - reconstructed)**2)
print(f"k={k}: MSE = {error:.2f}")
```

2.6 Norms — Distance & Regularization

L1 Norm (Manhattan): $||x||_1 = \sum |x_i|$ $\leftarrow$ LASSO regression
L2 Norm (Euclidean): $||x||_2 = \sqrt{\sum x_i^2}$ $\leftarrow$ Ridge regression
Lmax Norm (Infinity): $||x||_\infty = \max(|x_i|)$

```
import numpy as np

x = np.array([3, -4, 0, 2])

l1 = np.linalg.norm(x, ord=1)    # 3+4+0+2 = 9
l2 = np.linalg.norm(x, ord=2)    # sqrt(9+16+0+4) = 5.39
linf = np.linalg.norm(x, ord=np.inf) # max(3,4,0,2) = 4

print(f"L1 (LASSO): {l1}")
print(f"L2 (Ridge): {l2:.3f}")
print(f"Linf (Max): {linf}")

# Regularization in Linear Regression
from sklearn.linear_model import Ridge, Lasso

# Ridge uses L2 penalty: min ||Xθ-y||^2 + α||θ||_2^2
# Lasso uses L1 penalty: min ||Xθ-y||^2 + α||θ||_1
```

□ PART 3: CALCULUS

Optimization ka Engine

3.1 Derivatives — Rate of Change

Yeh samjho: "Ek point pe function kitna steep hai?"

Rules (Yaad Karo!)

Power Rule: $d/dx[x^n] = nx^{n-1}$
Constant: $d/dx[c] = 0$
Sum: $d/dx[f+g] = f' + g'$
Product: $d/dx[fg] = f'g + fg'$
Chain Rule: $d/dx[f(g(x))] = f'(g(x)) \times g'(x)$ $\leftarrow$ Backprop ka core!
Log: $d/dx[\ln(x)] = 1/x$
Exp: $d/dx[e^x] = e^x$

```
from sympy import symbols, diff, exp, log, sin, cos, simplify
```

```
x = symbols('x')
```

```
# Basic derivatives
f1 = x**3 + 2*x**2 - 5*x + 3
print(f"f(x) = {f1}")
print(f"f'(x) = {diff(f1, x)}")    # 3x^2 + 4x - 5
```

```
# Chain rule example: f(g(x)) = (x^2+1)^3
g = x**2 + 1
```

```
f = g**3
print(f"\nf(x) = {f}")
print(f"f'(x) = {diff(f, x)}")          #  $6x(x^2+1)^2$ 

# Sigmoid function (used in neural networks!)
def sigmoid(x_val):
    import numpy as np
    return 1 / (1 + np.exp(-x_val))

# Derivative of sigmoid:  $\sigma'(x) = \sigma(x)(1 - \sigma(x))$ 
sigma = 1 / (1 + exp(-x))
d_sigma = diff(sigma, x)
print(f"\nSigmoid derivative: {simplify(d_sigma)}")
```

3.2 Partial Derivatives — Multi-variable Calculus

ML mein: Model mein multiple parameters hote hain — har parameter ke liye partial derivative chahiye

```
f(x,y) = x2 + 3xy + y2

∂f/∂x = 2x + 3y    (y ko constant maano)
∂f/∂y = 3x + 2y    (x ko constant maano)

from sympy import symbols, diff

x, y, w1, w2, b = symbols('x y w1 w2 b')

# Neural network loss (simplified)
# Output:  $\hat{y} = w1*x + w2*y + b$ 
# Loss:  $L = (\hat{y} - y\_true)^2$ 

y_true = 3.0
y_hat = w1*x + w2*y + b
loss = (y_hat - y_true)**2

# Partial derivatives (gradients)
dL_dw1 = diff(loss, w1)
dL_dw2 = diff(loss, w2)
dL_db = diff(loss, b)

print("∂L/∂w1 =", dL_dw1)
print("∂L/∂w2 =", dL_dw2)
print("∂L/∂b  =", dL_db)
```

3.3 Gradient — Direction of Steepest Ascent

Gradient: Saare partial derivatives ka vector

$\nabla f = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]$

Gradient points in direction of MAXIMUM INCREASE
So to MINIMIZE, go in OPPOSITE direction → Gradient Descent!

3.4 Gradient Descent — □ ML ka MOST Important Concept

Intuition: Ek hill pe aankhein band karke neeche utarna — har step mein steepest slope pe chalte ho

```

Algorithm:
     $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \times \nabla L(\theta)$ 

Where:
     $\theta$  = parameters (weights)
     $\alpha$  = learning rate (step size)
     $\nabla L(\theta)$  = gradient of loss function

import numpy as np
import matplotlib.pyplot as plt

# Gradient Descent from scratch on a simple loss function
# Loss:  $L(\theta) = (\theta - 5)^2 + 3$ 

def loss(theta):
    return (theta - 5)**2 + 3

def gradient(theta):
    return 2 * (theta - 5) # dL/dθ

# Hyperparameters
theta = 0.0 # Start from 0
learning_rate = 0.1
n_iterations = 50

history = []
for i in range(n_iterations):
    grad = gradient(theta)
    theta = theta - learning_rate * grad
    loss_val = loss(theta)
    history.append((theta, loss_val))
    if i % 10 == 0:
        print(f"Iter {i:3d}: θ={theta:.4f}, Loss={loss_val:.4f}")

print(f"\nFinal θ = {theta:.4f} (True minimum at θ=5)")

# Effect of Learning Rate
plt.figure(figsize=(12, 4))
for lr in [0.01, 0.1, 0.9]:
    theta_temp = 0.0
    losses = []
    for _ in range(50):
        theta_temp -= lr * gradient(theta_temp)
        losses.append(loss(theta_temp))
    plt.plot(losses, label=f"lr={lr}")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Effect of Learning Rate on Gradient Descent")
plt.legend()
plt.savefig('gradient_descent.png')

```

Types of Gradient Descent

Type	Data Used	Update Frequency	Use Case
Batch GD	Full dataset	Once per epoch	Small datasets
Stochastic GD (SGD)	1 sample	Every sample	Online learning
Mini-batch GD	Batch (32-256)	Every batch	Standard in ML

```

import numpy as np

# Mini-batch Gradient Descent for Linear Regression
np.random.seed(42)
n = 1000
X = np.random.randn(n, 1)

```

```

y = 3 * X.squeeze() + 4 + np.random.randn(n) * 0.5

# Add bias term
X_b = np.c_[np.ones((n, 1)), X]

# Initialize
theta = np.zeros(2)
lr = 0.01
batch_size = 32
epochs = 100

for epoch in range(epochs):
    # Shuffle data
    indices = np.random.permutation(n)
    X_shuffled, y_shuffled = X_b[indices], y[indices]

    for start in range(0, n, batch_size):
        X_batch = X_shuffled[start:start+batch_size]
        y_batch = y_shuffled[start:start+batch_size]

        # Compute gradient
        predictions = X_batch @ theta
        errors = predictions - y_batch
        gradient = (2/batch_size) * X_batch.T @ errors

        # Update parameters
        theta -= lr * gradient

    if epoch % 20 == 0:
        loss = np.mean((X_b @ theta - y)**2)
        print(f"Epoch {epoch}:  $\theta_0$ ={theta[0]:.2f},  $\theta_1$ ={theta[1]:.2f}, Loss={loss:.4f}")

print(f"\nFinal: Intercept≈{theta[0]:.2f} (true: 4), Slope≈{theta[1]:.2f} (true: 3)")

```

3.5 Chain Rule — Backpropagation ka Foundation

Yeh neural networks ke training ka core hai!

For composed functions: $L(w) = \text{loss}(\text{output}(\text{hidden}(\text{input}(w))))$

$\partial L / \partial w = \partial L / \partial \text{output} \times \partial \text{output} / \partial \text{hidden} \times \partial \text{hidden} / \partial w$

← This is backpropagation!

```
import numpy as np
```

```
# Simple 2-layer neural network forward & backward pass
np.random.seed(42)
```

```
# Dummy data
X = np.array([[1.0, 2.0]])
y_true = np.array([[1.0]])
```

```
# Weights
W1 = np.random.randn(2, 3) # 2 inputs → 3 hidden
W2 = np.random.randn(3, 1) # 3 hidden → 1 output
```

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
```

```
def sigmoid_derivative(z):
    return sigmoid(z) * (1 - sigmoid(z))
```

```
# FORWARD PASS
z1 = X @ W1 # Hidden layer pre-activation
```

```

a1 = sigmoid(z1)          # Hidden layer activation
z2 = a1 @ W2              # Output pre-activation
y_pred = sigmoid(z2)      # Output

loss = 0.5 * (y_pred - y_true)**2

# BACKWARD PASS (Chain Rule!)
d_loss = y_pred - y_true      #  $\partial L / \partial y_{\text{pred}}$ 
d_z2 = d_loss * sigmoid_derivative(z2)  #  $\partial L / \partial z_2$ 
d_W2 = a1.T @ d_z2           #  $\partial L / \partial W_2$ 

d_a1 = d_z2 @ W2.T           #  $\partial L / \partial a_1$ 
d_z1 = d_a1 * sigmoid_derivative(z1)    #  $\partial L / \partial z_1$ 
d_W1 = X.T @ d_z1            #  $\partial L / \partial W_1$ 

# Update weights
lr = 0.1
W2 -= lr * d_W2
W1 -= lr * d_W1

print(f"Initial Loss: {loss[0][0]:.4f}")
print("Backprop done! Weights updated.")

```

3.6 Integration — Area Under Curve

ML mein use: Probability calculations, AUC-ROC curve

```

∫ f(x) dx = Area under f(x) curve

Fundamental Rules:
∫ x^n dx = x^(n+1)/(n+1) + C
∫ e^x dx = e^x + C
∫ 1/x dx = ln|x| + C

from scipy import integrate
import numpy as np

# Area under normal distribution curve between -1 and +1
from scipy.stats import norm

area, _ = integrate.quad(norm.pdf, -1, 1)
print(f"P(-1 < X < 1) = {area:.4f}")  # ~68%

area2, _ = integrate.quad(norm.pdf, -2, 2)
print(f"P(-2 < X < 2) = {area2:.4f}")  # ~95%

# AUC-ROC (Area Under ROC Curve)
from sklearn.metrics import roc_auc_score
import numpy as np

y_true = np.array([0, 0, 1, 1, 1, 0, 1, 0])
y_scores = np.array([0.1, 0.4, 0.35, 0.8, 0.7, 0.3, 0.9, 0.2])
auc = roc_auc_score(y_true, y_scores)
print(f"\nAUC-ROC Score: {auc:.4f}")  # 0.875 (good model!)

```

□ PART 4: PUTTING IT ALL TOGETHER

Real ML with Your Math Knowledge

4.1 End-to-End: Linear Regression from Scratch

```
import numpy as np
import matplotlib.pyplot as plt

class LinearRegression:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.losses = []

    def fit(self, X, y):
        n, features = X.shape
        X_b = np.c_[np.ones((n, 1)), X] # Add bias
        self.theta = np.zeros(features + 1)

        for epoch in range(self.epochs):
            # Forward pass (LINEAR ALGEBRA)
            y_pred = X_b @ self.theta

            # Compute loss (MSE)
            loss = np.mean((y_pred - y)**2)
            self.losses.append(loss)

            # Backward pass (CALCULUS - gradient descent)
            gradient = (2/n) * X_b.T @ (y_pred - y)
            self.theta -= self.lr * gradient

        return self

    def predict(self, X):
        X_b = np.c_[np.ones((len(X), 1)), X]
        return X_b @ self.theta

    def r2_score(self, X, y):
        y_pred = self.predict(X)
        ss_res = np.sum((y - y_pred)**2)
        ss_tot = np.sum((y - np.mean(y))**2)
        return 1 - ss_res/ss_tot

# Generate data
np.random.seed(42)
X = 2 * np.random.randn(100, 1)
y = 4 + 3 * X.squeeze() + np.random.randn(100)

# Train
model = LinearRegression(lr=0.01, epochs=500)
model.fit(X, y)

print(f"Intercept: {model.theta[0]:.3f} (true: 4)")
print(f"Slope: {model.theta[1]:.3f} (true: 3)")
print(f"R² Score: {model.r2_score(X, y):.4f}")
```

4.2 Logistic Regression (Probability + Calculus)

```
import numpy as np

class LogisticRegression:
    def __init__(self, lr=0.01, epochs=1000):
        self.lr = lr
        self.epochs = epochs

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))
```

```

def fit(self, X, y):
    n, features = X.shape
    X_b = np.c_[np.ones((n, 1)), X]
    self.theta = np.zeros(features + 1)

    for _ in range(self.epochs):
        # Forward pass
        z = X_b @ self.theta
        y_pred = self.sigmoid(z)

        # Cross-entropy loss gradient (PROBABILITY + CALCULUS)
        gradient = (1/n) * X_b.T @ (y_pred - y)
        self.theta -= self.lr * gradient

def predict_proba(self, X):
    X_b = np.c_[np.ones((len(X), 1)), X]
    return self.sigmoid(X_b @ self.theta)

def predict(self, X, threshold=0.5):
    return (self.predict_proba(X) >= threshold).astype(int)

# Test with binary classification
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=200, n_features=2,
                           n_redundant=0, random_state=42)

model = LogisticRegression(lr=0.1, epochs=1000)
model.fit(X, y)
accuracy = np.mean(model.predict(X) == y)
print(f"Accuracy: {accuracy:.4f}")

```

□ PART 5: RESOURCES & STUDY PLAN

5.1 Best Resources (Ranked for You)

Resource	Topic	Why Best	Link
3Blue1Brown	Linear Algebra, Calculus	Visual intuition, animations	youtube.com/@3blue1brown
Khan Academy	All topics	Free, structured, practice	khanacademy.org
Mathematics for ML Book	Complete	Cambridge standard, free PDF	mml-book.github.io
StatQuest (Josh Starmer)	Stats, ML	Clear explanations	youtube.com/@statquest
fast.ai	Practical ML	Top-down learning	fast.ai
Kaggle Learn	Applied ML	Hands-on practice	kaggle.com/learn

5.2 8-Week Study Plan (Tumare liye Customized)

Week 1-2: Probability & Statistics

- **Day 1-3:** Basic probability, events, conditional probability
- **Day 4-5:** Bayes' theorem (read 3 different explanations)
- **Day 6-7:** Code: simulate coin/dice, compute probabilities in Python

Mini Project: Build a spam classifier using Naive Bayes from scratch

Week 3: Distributions & Statistics

- **Day 1-3:** Normal, Binomial, Poisson distributions
- **Day 4-5:** Descriptive stats, z-scores, correlation
- **Day 6-7:** Hypothesis testing basics

Mini Project: Analyze a Kaggle dataset — find outliers, plot distributions

Week 4-5: Linear Algebra

- **Day 1-2:** Vectors, operations, dot product
- **Day 3-4:** Matrices, multiplication, inverse
- **Day 5:** Eigenvalues/eigenvectors (watch 3Blue1Brown first!)
- **Day 6-7:** SVD concept + PCA

Mini Project: Implement PCA from scratch using eigenvalues

Week 6: Calculus

- **Day 1-2:** Derivatives + rules (do 20 practice problems on paper)
- **Day 3-4:** Partial derivatives + gradient
- **Day 5-7:** Gradient descent — implement from scratch

Mini Project: Build linear regression with gradient descent from scratch

Week 7: Integration + Optimization

- **Day 1-3:** Basic integration, area under curve
- **Day 4-5:** Optimization concepts
- **Day 6-7:** Chain rule + backprop concept

Week 8: Apply Everything

- Build a neural network from scratch (no PyTorch/TensorFlow!)
- Implement backpropagation manually
- Compare with sklearn's implementations

5.3 Cheat Sheet — Saare Important Formulas

Probability

$$\begin{aligned}P(A|B) &= P(A \cap B) / P(B) \\P(A|B) &= P(B|A) \times P(A) / P(B) && \leftarrow \text{Bayes} \\P(A \cup B) &= P(A) + P(B) - P(A \cap B) \\P(A^c) &= 1 - P(A)\end{aligned}$$

Statistics

$$\begin{aligned}\text{Mean: } \mu &= \Sigma x / n \\ \text{Variance: } \sigma^2 &= \Sigma (x - \mu)^2 / n \\ \text{Std Dev: } \sigma &= \sqrt{\sigma^2} \\ \text{Z-score: } z &= (x - \mu) / \sigma \\ \text{Correlation: } \rho &= \text{Cov}(X, Y) / (\sigma_X \times \sigma_Y) \\ \text{Normal PDF: } f(x) &= (1 / \sigma \sqrt{2\pi}) \times e^{-(x - \mu)^2 / 2\sigma^2}\end{aligned}$$

Linear Algebra

Dot product: $a \cdot b = \sum a_i b_i = ||a|| ||b|| \cos(\theta)$
Matrix mult: $(AB)_{ij} = \sum A_{ik} B_{kj}$
L2 Norm: $||x|| = \sqrt{\sum x_i^2}$
Cosine Similarity: $\cos(\theta) = a \cdot b / (||a|| ||b||)$
Eigenvector: $Av = \lambda v$
SVD: $A = U \Sigma V^T$

Calculus

Power Rule: $d/dx[x^n] = nx^{n-1}$
Chain Rule: $d/dx[f(g(x))] = f'(g(x)) \times g'(x)$
Gradient: $\nabla f = [\partial f / \partial x_1, \dots, \partial f / \partial x_n]$
Gradient Descent: $\theta = \theta - \alpha \nabla L(\theta)$

5.4 AI/ML Math — Big Picture

AI/ML Model Training Loop:

1. Initialize weights (Linear Algebra: vectors)
↓
2. Forward Pass: compute predictions
 $y = \sigma(Wx + b)$ ← Matrix multiplication + sigmoid
↓
3. Compute Loss: measure error
 $L = -[y \times \log(\hat{y}) + (1-y) \times \log(1-\hat{y})]$ ← Probability (cross-entropy)
↓
4. Backward Pass: compute gradients
 $\partial L / \partial W = \dots$ ← Calculus (chain rule)
↓
5. Update weights:
 $W = W - \alpha \times \partial L / \partial W$ ← Gradient Descent
↓
6. Repeat until loss is small

Every step uses a different math topic!

*Guide made for someone who knows Python, JS/MERN, n8n, and automations.
Start with Part 1, code every example, and build projects after each week.
Math samajhne mein time lagta hai — consistency > intensity* □